

Ride Cancellation Prediction

The complete beginner's guide to your ML case study, and your viva

Case Study 66 · Machine Learning · Semester V · B.Tech CSE 2024-28

How to use this guide tonight

It is written for someone with **zero ML background**. Every term is explained in plain language before it is used, and every number comes from your own code (`src/train.py`), which was re-run to confirm the results.

If you only have 1 hour: read Chapter 1 (the 60-second pitch), Chapter 12 (the one-page cheat sheet) and Chapter 11 (the viva question bank).

If you have 3 hours: read everything in order. Each chapter follows one stage of the project: dataset → cleaning → choosing columns → charts → pipeline → models → evaluation → app → limitations.

Must-read before the demo: Chapter 10. It contains one issue in the code and one mismatch in the slides that an examiner could notice, and how to answer.

Contents

1. Your project in 60 seconds
2. Machine learning from zero
3. The dataset: where it comes from and what is in it
4. Framing the problem: what exactly are we predicting?
5. Preprocessing, step by step through the code
6. Selecting the best columns (feature selection and data leakage)
7. Exploratory data analysis: what the charts say
8. The ML pipeline: split, impute, encode, scale, balance
9. The two models: Logistic Regression and Random Forest
10. Evaluation: reading every number, choosing the winner, the app, limitations
11. Viva question bank (40 questions with answers)
12. One-page cheat sheet

1. Your project in 60 seconds

Memorise the shape of this, not the exact words. If you can say this confidently, you have already passed the first five minutes.

Say this when they ask "Explain your project"

"My project predicts whether a ride request will end up **cancelled**, at the exact moment the customer books it.

I used the public **Uber Request Data** dataset: 6,745 ride requests between Bangalore city and the airport in July 2016. Each request ended in one of three ways: trip completed (42%), no cars available (39%), or cancelled (18.7%).

I turned it into a **binary classification** problem: cancelled = 1, everything else = 0. I only used information that is known at booking time: the **pickup point** and the **request time** (hour, day, month, weekend). I deliberately removed Driver ID and Drop time, because they are only known after the booking, and using them would be **data leakage**, like seeing the answer before the exam.

I built a scikit-learn pipeline that fills missing values, one-hot encodes categories and scales numbers, then compared two models: **Logistic Regression** and **Random Forest**. Because only 18.7% of rows are cancellations, I used **balanced class weights** and judged the models with ROC-AUC, average precision and recall instead of plain accuracy.

Random Forest won, with **ROC-AUC 0.764** and **recall 82%**: it catches about 4 out of 5 real cancellations, with a precision of about 34%. So it works as an **early-warning system**, not as a final decision-maker. Finally I saved the model and built a **Streamlit web app** where you enter pickup point and time and get a cancellation probability."

The whole project on one picture

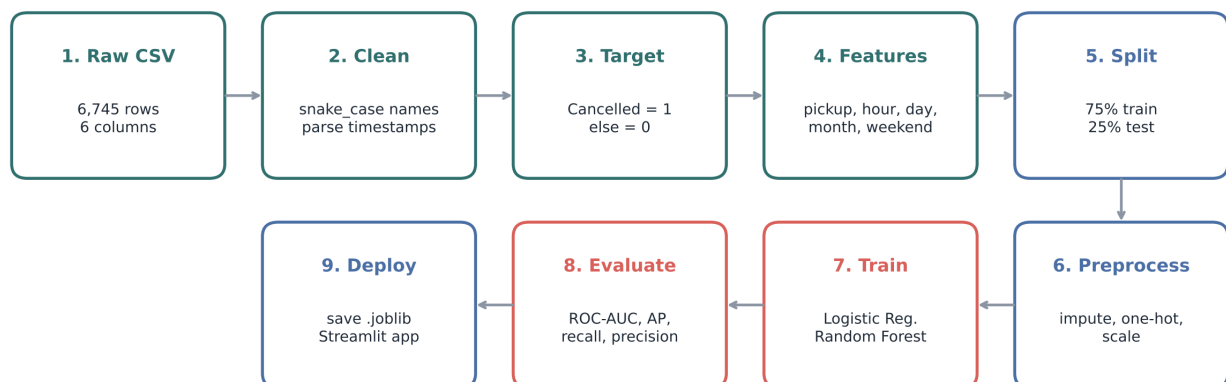


Figure 1. The nine stages of your project. Every chapter in this guide explains one or two of these boxes.

The five words you must be able to explain

Word	One-line meaning in your project
Classification	The answer is a category (cancelled / not cancelled), not a number like price.
Feature	An input column the model learns from: pickup_point, hour, day_of_week, month, is_weekend.
Target (label)	The column we want to predict: target_cancelled (1 or 0).
Data leakage	Accidentally giving the model information it would not have at prediction time. We avoided it by dropping Driver ID and Drop timestamp.
Recall	Of all real cancellations, how many did the model catch? Ours: 82%.

2. Machine learning from zero

Just enough theory to understand your own project. No maths degree needed.

2.1 What is machine learning?

Normal programming: **you** write the rules. "If pickup is City and hour is 7, then warn."

Machine learning: you give the computer **examples with answers**, and it **finds the rules itself**.

Think of how a new Uber dispatcher learns. Nobody gives them a rulebook. After watching thousands of bookings, they start to feel: "Morning city-to-airport rides get cancelled a lot." That feeling, learned from past examples, is exactly what an ML model builds, only with numbers instead of intuition.

2.2 Supervised learning

Your project is **supervised learning**: every row in the training data already has the correct answer (the Status column). The model studies the inputs and the known answers, then is tested on rows it has never seen.

Type	Meaning	Example
Supervised: classification	Predict a category	Your project: cancelled or not
Supervised: regression	Predict a number	Predict the fare in rupees
Unsupervised	No answers given; find groups	Group customers by behaviour

Yours is specifically **binary classification**, because there are exactly two classes: 1 (cancelled) and 0 (not cancelled).

2.3 Features and target

Picture your data as an Excel sheet. Each **row** is one ride request. The columns you feed in are the **features** (called X in code). The column you want to predict is the **target** (called y).

pickup_point	hour	day_of_week	month	is_weekend	→	target_cancelled
City	7	Monday	7	0	→	1 (cancelled)
Airport	18	Tuesday	7	0	→	0 (not cancelled)

Features = X (left side). Target = y (right side). The model learns a function: $y = f(X)$.

2.4 Training and testing

If you let a student practise on the exact questions that will come in the exam, a perfect score proves nothing. Same for models. So we **split** the data: the model learns from 75% (the **training set**, 5,058 rows) and we grade it on the other 25% (the **test set**, 1,687 rows), which it never saw during training. Test-set scores tell us how the model will behave on **new, future** bookings.

2.5 Overfitting and underfitting

- **Overfitting** = memorising instead of learning. Great on training data, poor on new data. Like a student who memorised last year's answers.
- **Underfitting** = too simple to capture the pattern. Poor everywhere.
- In your code, `min_samples_leaf=8` in the Random Forest is a guard against overfitting: no tree is allowed to make a rule based on fewer than 8 rides.

2.6 Probability and threshold

Your models do not just say "yes/no". They output a **probability** between 0 and 1, for example 0.85 = "85% chance of cancellation". To turn this into a yes/no decision, we pick a **threshold**. Your project uses **0.50**: 0.50 or more = "Higher risk", below = "Lower risk". Moving the threshold changes the balance between catching more cancellations and raising fewer false alarms (Chapter 10).

2.7 The tools (libraries) you used

Library	What it does in your project
pandas	Loads the CSV into a table (DataFrame), cleans columns, parses dates, creates new columns.
scikit-learn (sklearn)	The ML library: train/test split, imputation, encoding, scaling, both models, all metrics.
matplotlib + seaborn	Draws the charts in the figures/ folder.
joblib	Saves the trained model to a file (.joblib) so the app can load it without retraining.
Streamlit	Turns a Python script into a web app with dropdowns and buttons.

3. The dataset

Where the data comes from, what each column means, and what is missing.

3.1 Source

The dataset is the well-known **Uber Supply-Demand Gap** case-study dataset ("Uber Request Data.csv"), originally created for the IIIT Bangalore / upGrad data-science programme. It is publicly available on GitHub (dittakaviram/Data-Science repository) and mirrored on Kaggle. Your copy is `data/uber_request_data_raw.csv`.

- **Size:** 6,745 rows (ride requests) × 6 columns.
- **Place:** trips between Bangalore city and Kempegowda International Airport, in both directions.
- **Time:** five weekdays, Monday 11 July to Friday 15 July 2016 (see Chapter 10 for an important note about how the code reads these dates).
- **Duplicates:** none (checked in code: `raw.duplicated().sum() = 0`).

3.2 The six columns

Column	Meaning	Example	Used in model?
Request id	Unique number for each booking	619	No. Just an ID; it has no meaning for prediction.
Pickup point	Where the ride starts: City or Airport	Airport	Yes , as a feature
Driver id	Which driver accepted it	1	No. Not known at booking time (leakage)
Status	What finally happened	Trip Completed	Used to build the target
Request timestamp	Date and time of booking	11/7/2016 11:51	Yes , broken into hour, day, month, weekend
Drop timestamp	When the trip ended	11/7/2016 13:00	No. Only exists after the trip (leakage)

3.3 The three outcomes (Status column)

Status	Count	Share	What it means
Trip Completed	2,831	42.0%	Driver accepted and finished the trip.
No Cars Available	2,650	39.3%	No driver was free at all: a supply shortage.
Cancelled	1,264	18.7%	A driver was assigned, and then the ride was cancelled.
Total	6,745	100%	

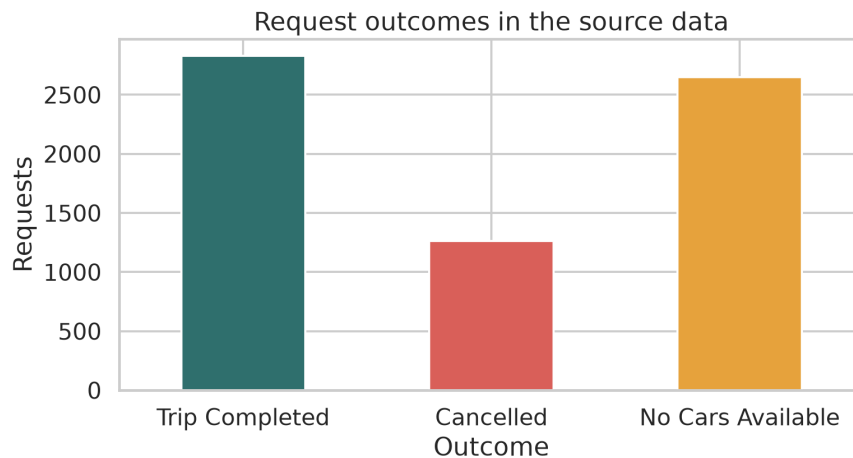


Figure 2. Outcome counts (figures/outcome_counts.png). Only 18.7% of requests are cancellations, so the classes are imbalanced.

3.4 Missing values, and why they are not a problem

Column	Missing	Why
Driver id	2,650	Exactly equals the number of "No Cars Available" rows. No car means no driver to record.
Drop timestamp	3,914	= 2,650 (no car) + 1,264 (cancelled). A trip that never happened has no drop time.
All other columns	0	Complete.

Key idea: "structural" missingness

These blanks are not random errors. They are missing **because of the outcome**. That is a big warning sign: if we used Driver id as a feature, the model would simply learn "Driver id is blank → No Cars Available", which is cheating. Since we never use these two columns as features, we do not need to fill them in.

4. Framing the problem

Deciding exactly what question the model answers is the most important design decision in any ML project.

4.1 The question

The ML question

"At the moment a customer books a ride, can we estimate the probability that this request will end up **Cancelled**, using only information available at that moment?"

4.2 Building the target column

The Status column has three values, but a binary classifier needs two. So we create a new column, `target_cancelled`:

```
df['target_cancelled'] = (df['status'].eq('Cancelled')).astype(int)
```

Read it as: "Is status equal to Cancelled? True becomes 1, False becomes 0." So:

Status	target_cancelled	Class name
Cancelled	1	Positive class (what we want to detect)
Trip Completed	0	Negative class
No Cars Available	0	Negative class

4.3 Why not treat "No Cars Available" as a cancellation too?

You will very likely be asked this. The answer: **they are different problems with different solutions.**

- **Cancelled** = a driver existed and was assigned, but the ride still fell through. A fix might be warning the rider, or offering the driver an incentive.
- **No Cars Available** = there was no driver at all. The fix is supply: bring more drivers to that area at that time.
- If we merged them, the model would answer a vaguer question ("will this request fail for any reason?") and hide which action is needed.

4.4 The "prediction moment"

Every ML system must define **when** it makes its prediction, because that decides which data is allowed. Your prediction moment is **the instant the booking is created**. At that moment we know where the customer is and what time it is. We do **not** yet know which driver will accept, or when the trip will end. This rule drives the column selection in Chapter 6.

5. Preprocessing, step by step

Raw data is never ready for a model. Here is every cleaning step in `src/train.py`, line by line, in plain English.

Step 1: Load the data

```
raw = pd.read_csv(DATA / 'uber_request_data_raw.csv')
df = raw.copy()
```

Read the CSV into a pandas DataFrame (an in-memory table). We work on a **copy** so the original raw data stays untouched for reporting.

Step 2: Clean the column names

```
df.columns = [c.strip().lower().replace(' ', '_') for c in df.columns]
```

"Pickup point" becomes `pickup_point`, "Request timestamp" becomes `request_timestamp`. This style is called **snake_case**. Spaces and capital letters in column names cause typing errors in code, so we standardise them.

Step 3: Convert text to real dates (datetime parsing)

```
df['request_timestamp'] = pd.to_datetime(df['request_timestamp'], errors='coerce')
```

In the CSV, "11/7/2016 11:51" is just text. `pd.to_datetime` converts it into a real date-time object so we can ask for the hour, weekday, etc. `errors='coerce'` means: if a value cannot be read, put a blank (NaT, "Not a Time") instead of crashing. **Important:** this line has a hidden problem with this dataset, explained in Chapter 10.7.

Step 4: Create the target

```
df['target_cancelled'] = (df['status'].eq('Cancelled')).astype(int)
```

Explained in Chapter 4.2.

Step 5: Feature engineering, creating new useful columns from the timestamp

Feature engineering means creating new columns that make patterns easier for the model to see. A full timestamp like "2016-07-11 07:32:00" is useless as-is (every value is unique). But the **hour** (7) is very useful, because cancellations depend on time of day.

```
df['hour'] = df['request_timestamp'].dt.hour # 0 to 23
df['day_of_week'] = df['request_timestamp'].dt.day_name() # 'Monday' ... 'Sunday'
df['day_num'] = df['request_timestamp'].dt.dayofweek # 0 = Monday ... 6 = Sunday
df['month'] = df['request_timestamp'].dt.month # 1 to 12
df['is_weekend'] = (df['day_num'] >= 5).astype(int) # 1 if Saturday/Sunday
df['time_band'] = pd.cut(df['hour'], bins=[-1,5,11,16,20,23],
                        labels=['Night','Morning','Afternoon','Evening','Late evening'])
```

New column	Type	Why it might help
hour	Number 0-23	Morning rush and late night behave differently.
day_of_week	Category	Mondays might differ from Fridays.
month	Number	Seasonal effects (in practice this data covers only one week).
is_weekend	0 or 1	Weekends could behave differently from weekdays.
time_band	Category	Groups hours into Night (0-5), Morning (6-11), Afternoon (12-16), Evening (17-20), Late evening (21-23). Used for analysis only, not fed to the model.

`pd.cut` is "binning": it chops a number range into labelled buckets. `bins=[-1,5,11,...]` means `(-1,5]` = Night, `(5,11]` = Morning, and so on.

Step 6: Save the prepared data

```
prepared_cols = ['request_id', 'pickup_point', 'status', 'request_timestamp', 'target_cancelled',
                 'hour', 'day_of_week', 'month', 'is_weekend', 'time_band']
df[prepared_cols].to_csv(DATA / 'uber_request_data_prepared.csv', index=False)
```

The cleaned table is saved as `data/uber_request_data_prepared.csv`, so anyone can audit what went into the model. Notice Driver id and Drop timestamp are not in it.

Where do imputation, encoding and scaling happen?

Those steps are also preprocessing, but they happen **inside the scikit-learn pipeline**, after the train/test split. That is deliberate and is explained in Chapter 8. The short version: anything that "learns" from data (like the median used to fill blanks) must learn only from training data.

6. Selecting the best columns

How we decided which columns go into the model. In ML this is called feature selection.

6.1 The two ways to choose columns

- **Domain / logic-based selection:** use common sense about the problem. "Is this column available at prediction time? Could it logically affect the outcome?" **This is the main method your project uses.**
- **Data-based selection:** measure how much each column actually helps the model (e.g. feature importance, correlation). Section 6.4 shows this for your model as supporting evidence.

6.2 The golden rule: no data leakage

Data leakage = the model gets information during training that it will not have in real life at the moment of prediction. The model then looks brilliant in testing and fails in the real world.

Exam analogy

Leakage is like a student who gets the answer key printed on the back of the question paper. They score 100%, but they learned nothing, and in the real exam (real bookings) they fail.

6.3 Column-by-column decision

Column	Known at booking?	Decision	Reason
Request id	Yes	Dropped	Just a serial number. It has no causal link to cancellation; using it could let the model memorise rows.
Pickup point	Yes	Kept	Known immediately. City vs Airport behaves very differently (30.4% vs 6.1% cancellation rate).
Driver id	No	Dropped	Leakage: no driver is assigned yet. It is also blank for every No Cars row, which would give away the outcome.
Status	No	Target	This IS the answer. It becomes target_cancelled.
Request timestamp	Yes	Kept (as derived features)	Transformed into hour, day_of_week, month, is_weekend. The raw timestamp itself is not used.
Drop timestamp	No	Dropped	Leakage: it is in the future and only exists for completed trips.
time_band	Yes	Not used in model	It is just a grouped version of hour. Hour already contains the same information, so it would be redundant.

Final feature list in the code:

```
features = ['pickup_point', 'hour', 'day_of_week', 'month', 'is_weekend']
X = df[features].copy()
y = df['target_cancelled']
```

6.4 Evidence: which columns does the model really use?

After training, we can measure each column's usefulness with **permutation importance**: shuffle one column randomly (destroying its information) and see how much the model's score drops. A big drop = an important column. This was computed for this guide on your saved Random Forest:

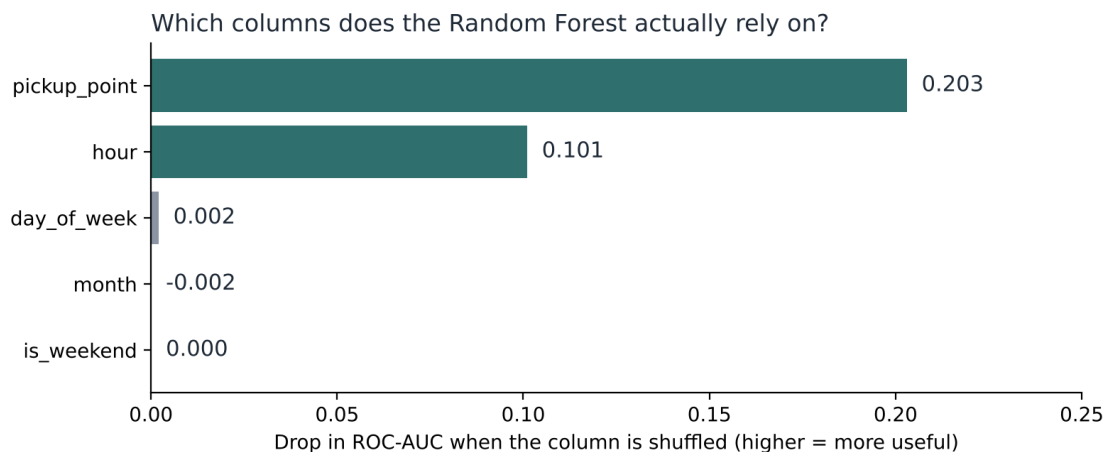


Figure 3. Permutation importance on the test set. Pickup point and hour carry almost all of the signal.

What to say about this

- **pickup_point** is the strongest signal, then **hour**. Together they basically are the model.
- **day_of_week, month, is_weekend** add almost nothing. That makes sense: the data covers only Monday to Friday of a single week, so **is_weekend** is always 0 and **month** never changes. A column that never varies cannot teach the model anything.
- They were kept because they are logically valid request-time features and would matter with a year of data. With this dataset, a simpler model with just **pickup_point + hour** would perform about the same. That is a good, honest viva answer.

7. Exploratory data analysis (EDA)

Before training a model, you look at the data with charts to understand patterns. This is EDA.

EDA answers: "Is there any pattern worth learning?" If cancellation rates were the same at every hour and location, no model could predict anything.

7.1 Cancellation rate by hour and pickup point

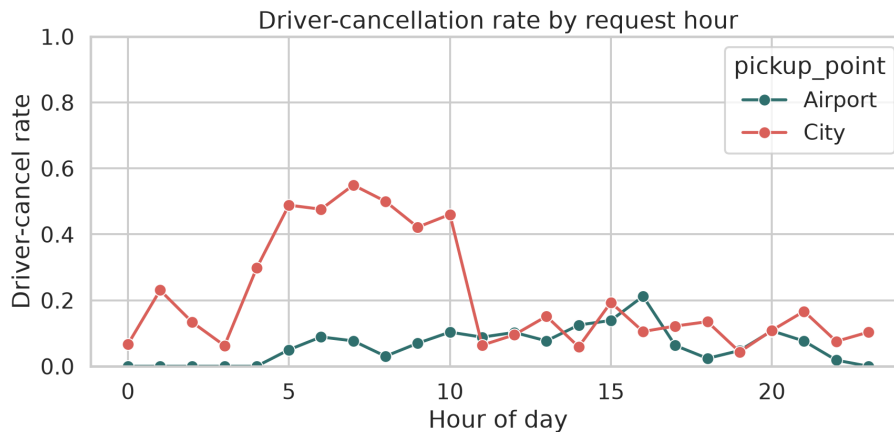


Figure 4. figures/cancel_rate_by_hour.png: share of requests cancelled at each hour, for City and Airport pickups.

- **City pickups in the early morning (roughly 4 am to 10 am) have very high cancellation rates.** These are mostly people going to the airport for morning flights.
- **Airport pickups have low cancellation rates** through most of the day.
- Likely reason (a hypothesis, not proven by data): a driver who takes a city→airport trip early in the morning may have to wait a long time at the airport or drive back empty, so some drivers cancel those trips.

7.2 All three outcomes by hour

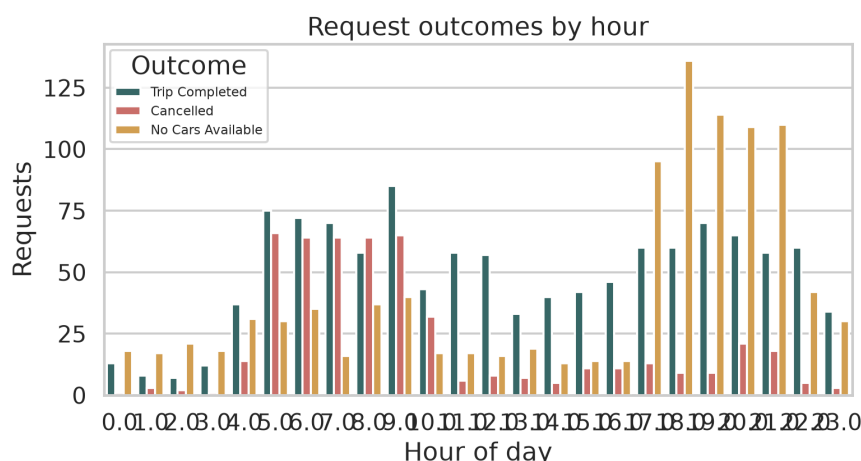


Figure 5. figures/outcomes_by_hour.png: completed, cancelled and no-car requests at each hour.

- **Morning peak:** lots of cancellations (red), mainly City requests.
- **Evening peak (about 5 pm to 10 pm):** "No Cars Available" (orange) dominates, mainly at the Airport, when many flights land and there are not enough cabs.

- This chart is why we kept "No Cars Available" separate: it peaks at a different time and place than cancellations, so it is a different problem.

7.3 Pickup point alone

Pickup point	Requests	Cancelled	Cancellation rate
City	3,507	1,066	30.4%
Airport	3,238	198	6.1%

A City request is about **5 times** more likely to be cancelled than an Airport request. This is why pickup_point is the most important feature.

EDA conclusions in one breath

"The target is imbalanced (18.7%), and it depends strongly on pickup point and hour of day. That justifies building a classifier from request-time features."

8. The ML pipeline

How the data is split, cleaned, converted to numbers and balanced before the models see it.

8.1 Train/test split, stratified

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
```

Parameter	Meaning
test_size=0.25	25% of rows go to the test set (1,687 rows), 75% to training (5,058 rows).
stratify=y	Keep the same share of cancellations (18.7%) in both sets. Without it, by bad luck the test set might get too few cancellations. Result: train has 948 cancellations, test has 316.
random_state=42	Fixes the random shuffle so the split is identical every run, which makes results reproducible . (42 is just a convention.)

8.2 Models only understand numbers

A model does arithmetic. It cannot multiply "City" by anything. So the pipeline transforms each column type differently using a **ColumnTransformer**, which applies different steps to different columns:

```
cat = ['pickup_point', 'day_of_week']      # text categories
num = ['hour', 'month', 'is_weekend']      # numbers

pre = ColumnTransformer([
    ('cat', Pipeline([('impute', SimpleImputer(strategy='most_frequent')),
                      ('onehot', OneHotEncoder(handle_unknown='ignore'))]), cat),
    ('num', Pipeline([('impute', SimpleImputer(strategy='median')),
                      ('scale', StandardScaler())]), num)])
```

a) Imputation: filling blanks

If a value is missing, `SimpleImputer` fills it: categories get the **most frequent** value (the mode), numbers get the **median** (middle value). The median is used instead of the mean because it is not pulled around by extreme values.

b) One-hot encoding: turning categories into 0/1 columns

`OneHotEncoder` turns one text column into several yes/no columns:

pickup_point	→	pickup_point_Airport	pickup_point_City
City	→	0	1
Airport	→	1	0

Why not just write City = 1, Airport = 2? Because the model would think Airport is "bigger" or "twice" City, which is meaningless. One-hot avoids inventing a fake order.

`handle_unknown='ignore'` means: if the app later sends a category never seen in training (e.g. "Friday"), just set all those columns to 0 instead of crashing.

c) Standard scaling: putting numbers on the same scale

StandardScaler converts each number to "how many standard deviations away from the average": $z = (x - \text{mean}) \div \text{std}$. Hour (0-23) and is_weekend (0-1) end up on comparable scales. This matters for Logistic Regression, which multiplies each feature by a weight; Random Forest does not need it but it does no harm.

8.3 Why put everything in a Pipeline?

```
pipe = Pipeline([('preprocess', pre), ('model', clf)])
pipe.fit(X_train, y_train)
```

- **Prevents leakage:** the median, the most frequent value, the mean and std for scaling are all learned **only from the training set** during fit. The test set is transformed using those training values, so no test information sneaks in.
- **One object does everything:** the saved file contains cleaning + model together. The Streamlit app just passes raw inputs ("City", 7, "Friday") and the pipeline handles the rest. No risk of preprocessing differently in the app.

8.4 Class imbalance and class_weight='balanced'

Only 18.7% of rows are cancellations. A lazy model could predict "not cancelled" for everyone and be right 81% of the time, while catching zero cancellations. To stop this, both models use `class_weight='balanced'`, which makes mistakes on the rare class cost more:

Class	Training rows	Weight = total ÷ (2 × class count)
Cancelled (1)	948	$5,058 \div (2 \times 948) \approx \mathbf{2.67}$
Not cancelled (0)	4,110	$5,058 \div (2 \times 4,110) \approx \mathbf{0.62}$

So missing a real cancellation is penalised about **4.3 times** more than a false alarm during training. This is why the model's recall is high (82%).

9. The two models

Your project compares a simple model with a more powerful one. This is standard good practice.

9.1 Logistic Regression (the baseline)

```
LogisticRegression(max_iter=1000, class_weight='balanced', random_state=42)
```

Despite the name, Logistic Regression is a **classification** model. It works in two steps:

- **Step 1, a score:** multiply each feature by a learned weight and add them up: $\text{score} = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + b$. A positive weight on `pickup_point_City` means "City pushes the risk up".
- **Step 2, squash into a probability:** pass the score through the **sigmoid** function, $p = 1 \div (1 + e^{-\text{score}})$, which always gives a value between 0 and 1.

Strengths: simple, fast, easy to explain (you can read the weights). **Weakness:** it assumes each feature has a straight-line, additive effect. It struggles with "City is risky *only* in the morning" type patterns (an **interaction**). `max_iter=1000` just gives the solver enough steps to finish learning.

9.2 Random Forest (the winner)

```
RandomForestClassifier(n_estimators=350, min_samples_leaf=8,  
                       class_weight='balanced', random_state=42, n_jobs=-1)
```

First, a decision tree

A decision tree is a flowchart of yes/no questions learned from data, like a game of 20 questions:

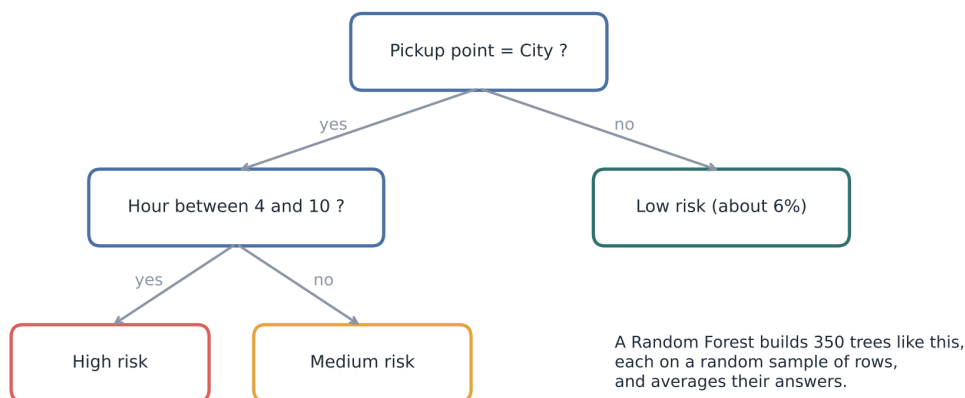


Figure 6. A simplified, illustrative tree. Real trees in your forest are deeper and learned automatically.

A single tree is easy to understand but unstable: small changes in data can produce a very different tree, and deep trees overfit.

Then, a forest of trees

- Build **350 trees** (`n_estimators=350`).

- Each tree is trained on a **random sample of rows** (drawn with replacement, called **bootstrapping**) and considers a **random subset of features** at each split.
- To predict, every tree gives its probability and the forest **averages** them. This combining is called **ensemble learning** (specifically, **bagging**).
- **Analogy:** asking 350 people who each saw slightly different evidence and taking the average opinion is more reliable than asking one expert.

Parameter	Value	Meaning
n_estimators	350	Number of trees. More trees = more stable, but slower.
min_samples_leaf	8	Each final leaf must contain at least 8 training rows. Stops trees making rules from 1-2 rides (prevents overfitting).
class_weight	balanced	Handles imbalance (Chapter 8.4).
random_state	42	Reproducible results.
n_jobs	-1	Use all CPU cores to train faster.

9.3 Why compare two models?

You should never just pick one fancy model. Logistic Regression is the **baseline**: if a complex model cannot beat a simple one, the complexity is not worth it. Here Random Forest beat it on ROC-AUC and average precision, which suggests the data has **non-linear interactions** (City × morning) that trees capture better.

10. Evaluation, the app, and limitations

What every number means, why Random Forest was chosen, and the honest weaknesses you should know before the viva.

10.1 The confusion matrix: the base of every metric

On the 1,687 test requests (316 real cancellations, 1,371 non-cancellations), at threshold 0.50, the Random Forest gave:

	Predicted: not cancelled	Predicted: cancelled
Actual: not cancelled (1,371)	863 True Negative (TN) correctly left alone	508 False Positive (FP) false alarm
Actual: cancelled (316)	57 False Negative (FN) missed cancellation	259 True Positive (TP) caught cancellation

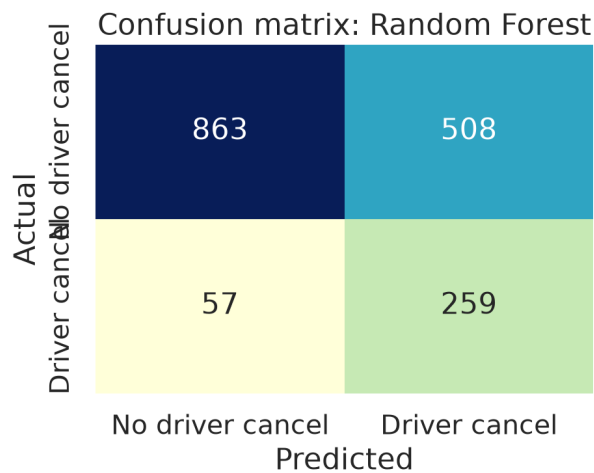


Figure 7. figures/confusion_matrix.png, the same numbers as a heatmap.

10.2 Every metric, with your numbers

Metric	Formula	Your RF value	Plain meaning
Accuracy	$(TP+TN) \div \text{all}$	$(259+863) \div 1,687 = \mathbf{66.5\%}$	Share of all predictions that were right.
Precision	$TP \div (TP+FP)$	$259 \div 767 = \mathbf{33.8\%}$	When the model raises an alarm, how often is it a real cancellation? About 1 in 3.
Recall (sensitivity)	$TP \div (TP+FN)$	$259 \div 316 = \mathbf{82.0\%}$	Of all real cancellations, how many did we catch? About 4 in 5.
F1 score	$2 \cdot P \cdot R \div (P+R)$	0.478	One number balancing precision and recall (their harmonic mean).

Metric	Formula	Your RF value	Plain meaning
ROC-AUC	Area under ROC curve	0.764	If you pick one random cancelled and one random non-cancelled request, the model gives the cancelled one a higher risk score 76.4% of the time. 0.5 = coin toss, 1.0 = perfect.
Average precision (PR-AUC)	Area under precision-recall curve	0.373	Ranking quality for the rare class. A random guess would score about 0.187 (the cancellation rate), so we are about 2× better than random.

Why accuracy is NOT the main metric (very common viva question)

A "dumb" model that always says "not cancelled" would get $1,371 \div 1,687 = 81.3\%$ **accuracy**, higher than our 66.5%, while catching **zero** cancellations. So with imbalanced data, accuracy is misleading. That is why we focus on recall, precision, ROC-AUC and average precision.

10.3 ROC and precision-recall curves

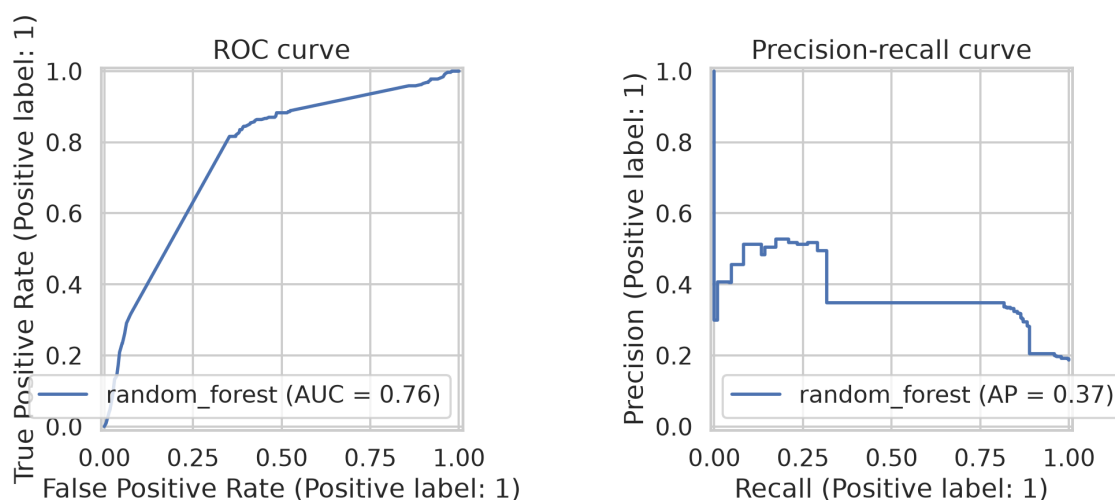


Figure 8. figures/evaluation_curves.png. Left: ROC curve (higher and further left is better). Right: precision-recall curve.

- **ROC curve:** plots true-positive rate (recall) against false-positive rate for every possible threshold. The diagonal line is random guessing; the further the curve bows toward the top-left, the better. Area under it = ROC-AUC = 0.764.
- **Precision-recall curve:** shows the trade-off: if you want to catch more cancellations (move right), precision drops. More useful than ROC when the positive class is rare.
- Both are **threshold-independent**: they judge the model's ranking ability, not one particular cut-off.

10.4 Model comparison and selection

Metric	Random Forest (selected)	Logistic Regression
ROC-AUC	0.764	0.752
Average precision	0.373	0.353
Precision	0.338	0.313
Recall	0.820	0.858
F1	0.478	0.458
Accuracy	0.665	0.620

```
metrics = pd.DataFrame(results).sort_values(['average_precision', 'roc_auc'], ascending=False)
best_name = metrics.iloc[0]['model']
joblib.dump(best_pipe, MODEL/'ride_cancellation_model.joblib')
```

Selection rule: sort by average precision, then ROC-AUC; the top model wins. Random Forest wins on 5 of 6 metrics. Logistic Regression has slightly higher recall, but only because it raises many more false alarms (596 vs 508), which is why its precision and accuracy are lower. The winning pipeline is saved with **joblib**.

10.5 The Streamlit app

app/streamlit_app.py loads the saved model and shows a form: pickup point, hour (slider), day, month. On submit:

```
row = pd.DataFrame([{'pickup_point': pickup_point, 'hour': hour, 'day_of_week': day,
                    'month': month, 'is_weekend': int(day in ['Saturday', 'Sunday'])}])
probability = float(model.predict_proba(row)[0, 1]) # probability of class 1
label = 'Higher risk' if probability >= 0.50 else 'Lower risk'
```

predict_proba returns two probabilities [not cancelled, cancelled]; [0, 1] picks the cancellation probability. The app is live at ride-cancellation-prediction.streamlit.app.

Demo plan: use these inputs (tested on your saved model)

- **High risk demo:** City, hour 7, Friday, July → about **85%** → "Higher risk". Explain: early-morning city pickups have the highest cancellation rate.
- **Low risk demo:** Airport, hour 18, Friday, July → about **6%** → "Lower risk".
- **Warning:** slide 8 of your presentation shows "Airport, 18:00, Friday, July → 68.4%, Higher risk", and evidence/viva_notes.md suggests demoing that input. Your real model gives about 6% for that input. If you demo it, the result will not match the slide. Either demo City 07:00 instead, or fix the slide to show City 07:00 → about 85%.

10.6 Limitations (say these before they ask)

- **Small, narrow data:** one week (five weekdays) in July 2016, one city. No weekends, no seasons.
- **Missing context:** no traffic, weather, fare/surge, driver distance, wait time, or customer history. These would likely improve predictions a lot.
- **Correlation, not causation:** the model finds that morning City requests get cancelled, not *why*.
- **Random split, not time-based:** a stricter test would train on earlier days and test on later days.
- **Fixed 0.50 threshold:** in production it should be tuned to the business cost of a missed cancellation vs a false alarm.
- **Precision is modest (34%):** two of every three alarms are false, so the score should support decisions (alerts, staffing), never punish drivers or riders.

10.7 A known issue in the code: date parsing (read this carefully)

The raw CSV mixes **two date formats**:

```
11/7/2016 11:51      <- day/month/year with slashes, no seconds
13-07-2016 08:18:13 <- day-month-year with dashes, with seconds
```

`pd.to_datetime(..., errors='coerce')` guesses one format from the first rows. With the pandas version used, it read "11/7/2016" as **7 November** (month first) and could not read the dash-format rows at all, so it silently turned them into blanks.

What happened	Number
Timestamps that became blank (NaT)	4,071 of 6,745 (60%)
Dates actually in the data	Mon 11 to Fri 15 July 2016
Months the code ended up with	11 (Nov) and 12 (Dec), plus blanks
What filled the blanks	SimpleImputer: median hour, most frequent day and month

So for 60% of rows the model only really saw the pickup point; the hour was filled in with a typical value. The model still works because pickup point is the strongest signal and 40% of rows had correct hours. This is also why the report says "July and November".

The fix, and what it changes

One-line fix in `src/train.py`:

```
pd.to_datetime(df['request_timestamp'], format='mixed', dayfirst=True)
```

With that fix, all 6,745 timestamps parse correctly, and re-running the same code gives a better Random Forest: **ROC-AUC 0.801** (was 0.764), **average precision 0.451** (was 0.373), precision 0.433, recall 0.756, F1 0.551, accuracy 0.769.

How to handle it in the viva

If an examiner asks about the months, the dates, or "why July and November?", do not bluff. Say:

"The raw file mixes two date formats. My parser only read about 40% of timestamps correctly and the rest were filled by the imputer, so the time features are weaker than they should be. I found this during review. With `format='mixed'`, `dayfirst=True` every row parses, the data turns out to be 11 to 15 July, and ROC-AUC improves from 0.76 to 0.80. It actually confirms that hour of day is a real signal."

Examiners usually respect a student who knows the weaknesses of their own project. It shows you understand the data, not just the code.

11. Viva question bank

40 likely questions, grouped by topic. Read the answers out loud once; that is how they stick.

A. Problem and data

Q1. What is your project about?

Predicting whether a ride request will be cancelled, at the moment of booking, using only pickup point and request time. Binary classification on the Uber Request Data dataset.

Q2. Where is the dataset from?

The public Uber Supply-Demand Gap dataset from the IIIT Bangalore / upGrad case study, available on GitHub and Kaggle. 6,745 requests, 6 columns, Bangalore city ↔ airport, July 2016.

Q3. Is this classification or regression? Why?

Classification, specifically binary, because the output is a category: cancelled (1) or not (0). Regression would predict a continuous number like fare.

Q4. Supervised or unsupervised?

Supervised, because every training row has a known answer in the Status column.

Q5. What are your features and target?

Features: pickup_point, hour, day_of_week, month, is_weekend. Target: target_cancelled, 1 if Status is Cancelled, else 0.

Q6. What is the class distribution?

Trip Completed 42.0%, No Cars Available 39.3%, Cancelled 18.7%. For the model: 18.7% positive, 81.3% negative, so it is imbalanced.

Q7. Why didn't you merge "No Cars Available" with "Cancelled"?

They need different actions. No Cars is a supply shortage (add drivers); Cancelled means a driver was assigned but the ride still failed. Merging hides the difference. The EDA also shows they peak at different times and places.

Q8. Were there missing values? How did you handle them?

Driver id is missing in 2,650 rows and Drop timestamp in 3,914. Both are structural: they are missing because the trip did not happen. We do not use either as a feature, so no filling was needed. Inside the pipeline, SimpleImputer fills any remaining blanks in features (median for numbers, most frequent for categories).

Q9. Any duplicates?

No, zero duplicate rows.

B. Preprocessing and features

Q10. What preprocessing did you do?

Renamed columns to snake_case, parsed timestamps, created the binary target, extracted hour, day_of_week, month and is_weekend, then in the pipeline: imputation, one-hot encoding for categories, standard scaling for numbers.

Q11. What is feature engineering? Give an example.

Creating new informative columns from existing ones. Example: extracting hour from the request timestamp, since the full timestamp is unique per row but the hour shows the daily pattern.

Q12. What is data leakage? How did you avoid it?

Using information during training that would not be available at prediction time. We dropped Driver id and Drop timestamp because they are only known after booking. We also put preprocessing inside a Pipeline so imputation and scaling learn only from training data.

Q13. How did you select features?

Mainly by logic: keep only columns known at booking time with a plausible link to cancellation. Permutation importance afterwards confirmed that pickup_point and hour carry nearly all the signal.

Q14. Why did you drop Request id?

It is just a serial number with no real relationship to cancellation. Including it could let the model memorise rows instead of learning patterns.

Q15. What is one-hot encoding and why use it?

It converts a category into separate 0/1 columns (pickup_point_City, pickup_point_Airport). Models need numbers, and simple label numbers like City=1, Airport=2 would imply a false order.

Q16. What does StandardScaler do?

Rescales each numeric feature to mean 0 and standard deviation 1 using $z = (x - \text{mean}) \div \text{std}$, so features are on comparable scales. Important for Logistic Regression, not needed for Random Forest.

Q17. Why median and not mean for imputation?

The median is robust to outliers; extreme values do not pull it.

Q18. What is ColumnTransformer / Pipeline?

ColumnTransformer applies different preprocessing to different columns. Pipeline chains preprocessing and the model into one object, so fit and predict always apply the same steps, which avoids leakage and simplifies deployment.

Q19. You created time_band. Why is it not a model feature?

It is a grouped version of hour, so it is redundant. It was kept in the prepared data for analysis and charts.

Q20. day_of_week, month and is_weekend seem useless. Why keep them?

They are valid request-time features in general, but this dataset covers only five weekdays of one month, so they barely vary and add almost nothing here. With a full year of data they would matter. A model with only pickup_point and hour would perform about the same.

C. Models

Q21. Which models did you use and why?

Logistic Regression as a simple, interpretable baseline, and Random Forest as a stronger non-linear model that can capture interactions like City × morning.

Q22. How does Logistic Regression work?

It computes a weighted sum of the features and passes it through the sigmoid function, $1 \div (1 + e^{-z})$, to get a probability between 0 and 1.

Q23. Why is it called "regression" if it classifies?

Because internally it regresses (fits a linear equation to) the log-odds of the class. The output probability is then thresholded to classify.

Q24. How does Random Forest work?

It trains many decision trees (350 here), each on a bootstrap sample of rows and random subsets of features, then averages their predictions. Averaging many diverse trees reduces overfitting and variance.

Q25. What is a decision tree?

A flowchart of yes/no questions on features (e.g. "Is pickup City?", "Is hour < 10?") that ends in a prediction at each leaf.

Q26. What do `n_estimators=350` and `min_samples_leaf=8` mean?

350 trees in the forest; each leaf must contain at least 8 training samples, which stops trees from making rules from very few rides and so reduces overfitting.

Q27. What is `class_weight='balanced'`?

It gives the rare class more weight in training, inversely proportional to its frequency. Here a cancellation counts about 2.67 and a non-cancellation about 0.62, so the model cannot ignore cancellations.

Q28. Other ways to handle imbalance?

Oversampling the minority class (e.g. SMOTE), undersampling the majority class, or adjusting the decision threshold.

Q29. Did you tune hyperparameters?

No formal grid search; reasonable values were chosen. A next step would be GridSearchCV or RandomizedSearchCV with cross-validation.

Q30. What is `random_state=42`?

A fixed seed for randomness so the split and the forest are identical every run, making results reproducible.

D. Evaluation

Q31. Why didn't you use accuracy as the main metric?

Because of imbalance. Predicting "not cancelled" for everyone gives 81.3% accuracy but catches no cancellations. Recall, precision, ROC-AUC and average precision show performance on the rare class.

Q32. Explain your confusion matrix.

On 1,687 test rows: 259 cancellations correctly caught (TP), 57 missed (FN), 508 false alarms (FP), 863 correctly left alone (TN).

Q33. What do precision 0.338 and recall 0.820 mean?

Recall: we catch 82% of real cancellations. Precision: about one in three alarms is a real cancellation. We chose to favour recall because missing a cancellation is worse than an extra alert.

Q34. What is ROC-AUC 0.764?

There is a 76.4% chance the model ranks a random cancelled request above a random non-cancelled one. 0.5 is random, 1.0 is perfect.

Q35. What is average precision 0.373?

The area under the precision-recall curve. A random model would score about 0.187 (the positive rate), so ours is roughly twice as good as random at ranking cancellations.

Q36. Why did you pick Random Forest when Logistic Regression has higher recall?

Random Forest is better on ROC-AUC, average precision, precision, F1 and accuracy. LR's slightly higher recall comes from raising many more false alarms (596 vs 508). The selection rule was average precision, then ROC-AUC.

Q37. How would you improve precision?

Raise the threshold above 0.5, add better features (driver distance, wait time, traffic, weather, surge), get more data, tune hyperparameters, and try gradient boosting (XGBoost/LightGBM).

E. Deployment, limits, future**Q38. How does your app work?**

Streamlit loads the saved joblib pipeline. The user picks pickup point, hour, day and month; the app builds a one-row DataFrame, calls `predict_proba`, shows the probability, and labels it Higher risk if ≥ 0.50 .

Q39. What are the limitations?

One week of data in one city, no weekends; no traffic, weather, fare or driver-location features; correlation not causation; random rather than time-based split; fixed threshold; and the date-parsing issue described in Chapter 10.7.

Q40. How would this be used in real life?

As an early-warning signal: flag high-risk bookings to offer driver incentives, give riders a heads-up, or plan driver supply for morning airport runs. It should never be used to penalise drivers or riders, because precision is only about 34%.

12. One-page cheat sheet

Read this five minutes before you walk in.

Item	Value
Dataset	Uber Request Data (IIIT-B / upGrad), public on GitHub & Kaggle
Size	6,745 rows × 6 columns, no duplicates
Outcomes	Completed 2,831 (42.0%) · No Cars 2,650 (39.3%) · Cancelled 1,264 (18.7%)
Problem type	Supervised, binary classification
Target	target_cancelled = 1 if Status == Cancelled, else 0
Features (5)	pickup_point, hour, day_of_week, month, is_weekend
Dropped (leakage)	Driver id (not known yet), Drop timestamp (future). Request id: just an ID
Strongest features	pickup_point, then hour. City 30.4% vs Airport 6.1% cancel rate
Split	75/25 stratified, random_state=42 → 5,058 train / 1,687 test (316 cancellations in test)
Preprocessing	Categories: mode impute → OneHotEncoder · Numbers: median impute → StandardScaler
Imbalance	class_weight='balanced' (cancel ≈ 2.67, not-cancel ≈ 0.62)
Models	Logistic Regression (baseline) vs Random Forest (350 trees, min_samples_leaf=8)
Winner	Random Forest, chosen by average precision then ROC-AUC
RF results	ROC-AUC 0.764 · AP 0.373 · Precision 0.338 · Recall 0.820 · F1 0.478 · Accuracy 0.665
LR results	ROC-AUC 0.752 · AP 0.353 · Precision 0.313 · Recall 0.858 · F1 0.458 · Accuracy 0.620
Confusion matrix (RF)	TN 863 · FP 508 · FN 57 · TP 259
Accuracy trap	"Always not cancelled" = 81.3% accuracy, 0 cancellations caught
Key EDA insight	Morning City pickups → cancellations. Evening Airport → No Cars Available
App	Streamlit + joblib; threshold 0.50; demo City 07:00 (~85%) and Airport 18:00 (~6%)
Known issue	Mixed date formats → 60% timestamps unparsed. Fix: format='mixed', dayfirst=True → ROC-AUC 0.80
Top limitations	1 week, 1 city; no traffic/weather/fare; correlation ≠ causation; fixed threshold

Three sentences that save you when you blank out

1. "Only information available at booking time is used; that is why Driver ID and Drop time are excluded."
2. "The data is imbalanced, so I used balanced class weights and looked at recall and ROC-AUC rather than accuracy."
3. "Random Forest catches 82% of cancellations; it is an early-warning tool, not a decision-maker."